

Esempio

- **Problema:** Consideriamo un programma **prog1** che scrive su standard output. Come si puo' fare in modo che l'output venga visualizzato una pagina alla volta, senza pero' modificare il programma stesso?
- **Soluzione:** si scrive un altro programma che:
 - crea un pipe e poi genera un processo child mediante `fork`
 - nel codice del parent chiude l'estremo di read del pipe e lo `stdout`, e riassegna mediante **`dup2`** il fd dello **`stdout`** (**1**) sull'estremo di write del pipe
 - nel codice del child chiude l'estremo di write del pipe e lo `stdin`, e riassegna mediante **`dup2`** il fd dello **`stdin`** (**0**) sull'estremo di read del pipe
 - il parent mediante **`exec`** lancia il programma **`prog1`**
 - il child mediante **`exec`** lancia un programma tipo di paginazione dell'output tipo **`more`** o **`less`**

Pipe e Dup

- Nell'esempio precedente lettura e scrittura direttamente su pipe descriptor
- Interessante è l'uso della duplicazione dei pipe descriptors su stdin o stout
- Es. quando programmi scrivono o leggono su stdin o stdout

Pipe tra due programmi

Per associare lo stdout o lo stdin di un programma, rispettivamente, al lato in scrittura o a quello in lettura di una pipe, si può utilizzare la funzione seguente:

```
#include <unistd.h>

int dup2 ( int filedes, int filedes2 );
```

- duplica il descrittore di file *filedes*, nel nuovo descrittore *filedes2* ;
- se *filedes2* è aperto, esso viene chiuso prima della duplicazione;
- restituisce il nuovo descrittore *filedes2* in caso di successo, **-1** altrimenti.

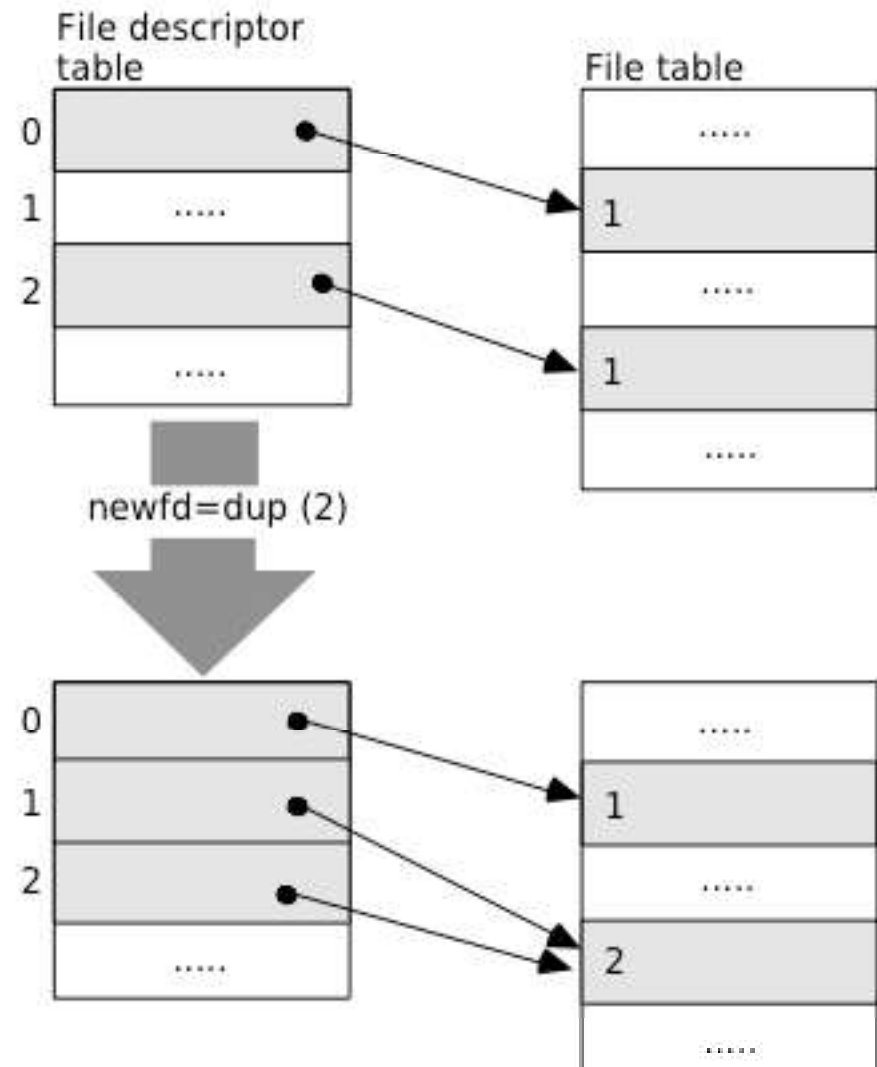
Duplicazione (vedi File System)

- Un file descriptor esistente viene duplicato da una delle seguenti funzioni:

- `int dup(int filedes);`

- `int dup2(int filedes,
int filedes2);`

- Entrambe le funzioni “duplicano” un file descriptor, ovvero creano un nuovo file descriptor che punta alla stessa file table entry del file descriptor originario
- Nella file table entry c’è un campo che registra il numero di file descriptor che la “puntano”



Duplicazione (vedi File System)

- Funzione `dup`
 - Seleziona il più basso file descriptor libero della tabella dei file descriptor
 - Assegna la nuova file descriptor entry al file descriptor selezionato
 - Ritorna il file descriptor selezionato
- Funzione `dup2`
 - Con **`dup2`**, specifichiamo il valore del nuovo descrittore come argomento **`filesdes2`**
 - Se **`filesdes2`** è già aperto, viene chiuso e sostituito con il descrittore duplicato
 - Ritorna il file descriptor selezionato

Pipe tra due programmi

Per associare lo stdout o lo stdin di un programma, rispettivamente, al lato in scrittura o a quello in lettura di una pipe, si può utilizzare la funzione seguente:

```
#include <unistd.h>

int dup2 ( int filedes, int filedes2 );
```

- duplica il descrittore di file *filedes*, nel nuovo descrittore *filedes2* ;
- se *filedes2* è aperto, esso viene chiuso prima della duplicazione;
- restituisce il nuovo descrittore *filedes2* in caso di successo, **-1** altrimenti.

Esempio

Legge un file indicato sulla linea di comando e lo visualizza utilizzando un pager.

```
#define DEF_PAGER    "/bin/more"    /* default pager program */

int main(int argc, char *argv[])
{
    if (argc != 2)
        err_quit("usage: a.out <pathname>");

    if ((fp = fopen(argv[1], "r")) == NULL)
        err_sys("can't open %s", argv[1]);
    if (pipe(fd) < 0)
        err_sys("pipe error");
```

Esempio

```
if ((pid = fork()) < 0) {
    perror("fork error");
} else if (pid > 0) { /* padre */
    close(fd[0]);      /* chiude la pipe il lettura */

    /* il padre copia il file argv[1] sulla pipe */

    while (fgets(line, MAXLINE, fp) != NULL) {
        n = strlen(line);
        if (write(fd[1], line, n) != n)
            perror("write error to pipe");
    }
    if (ferror(fp))
        perror("fgets error");

    close(fd[1]); /* chiude la pipe in scrittura */

    if (waitpid(pid, NULL, 0) < 0)
        err_sys("waitpid error");
    exit(0);
}
```


Esempio

```
else {
    /* figlio */
    close(fd[1]); /* chiude la pipe in scrittura*/
    if (fd[0] != STDIN_FILENO) { /* controlla che non sia già ok*/
        if (dup2(fd[0], STDIN_FILENO) != STDIN_FILENO)
            perror("dup2 error to stdin");
        close(fd[0]); /* non necessario dopo dup2*/
    }

    /* costruisce gli argomenti per execl() */
    if ((pager = getenv("PAGER")) == NULL)
        pager = DEF_PAGER;
    if ((argv0 = strrchr(pager, '/')) != NULL)
        argv0++; /* puntatore ad ultimo '/' in pager,
                  estraе il nome del comando */
    else
        argv0 = pager; /* non ci sono "/" in pager */

    if (execl(pager, argv0, (char *)0) < 0)
        err_sys("execl error for %s", pager);
}
exit(0);
```

Pipe tra due programmi

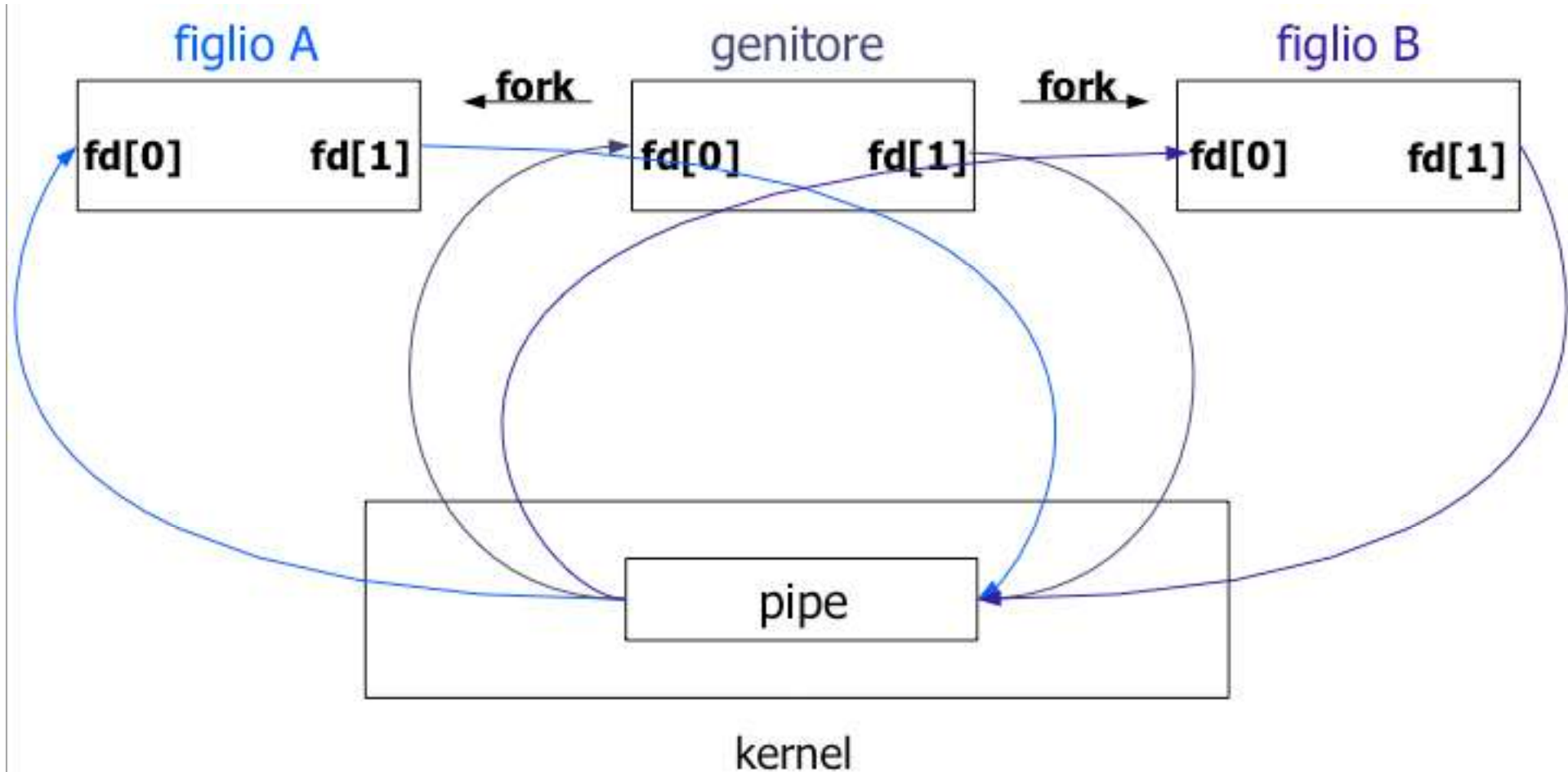
Per realizzare una pipe tra due programmi, come

```
ls | grep old
```

la sequenza di eventi è precisamente la seguente:

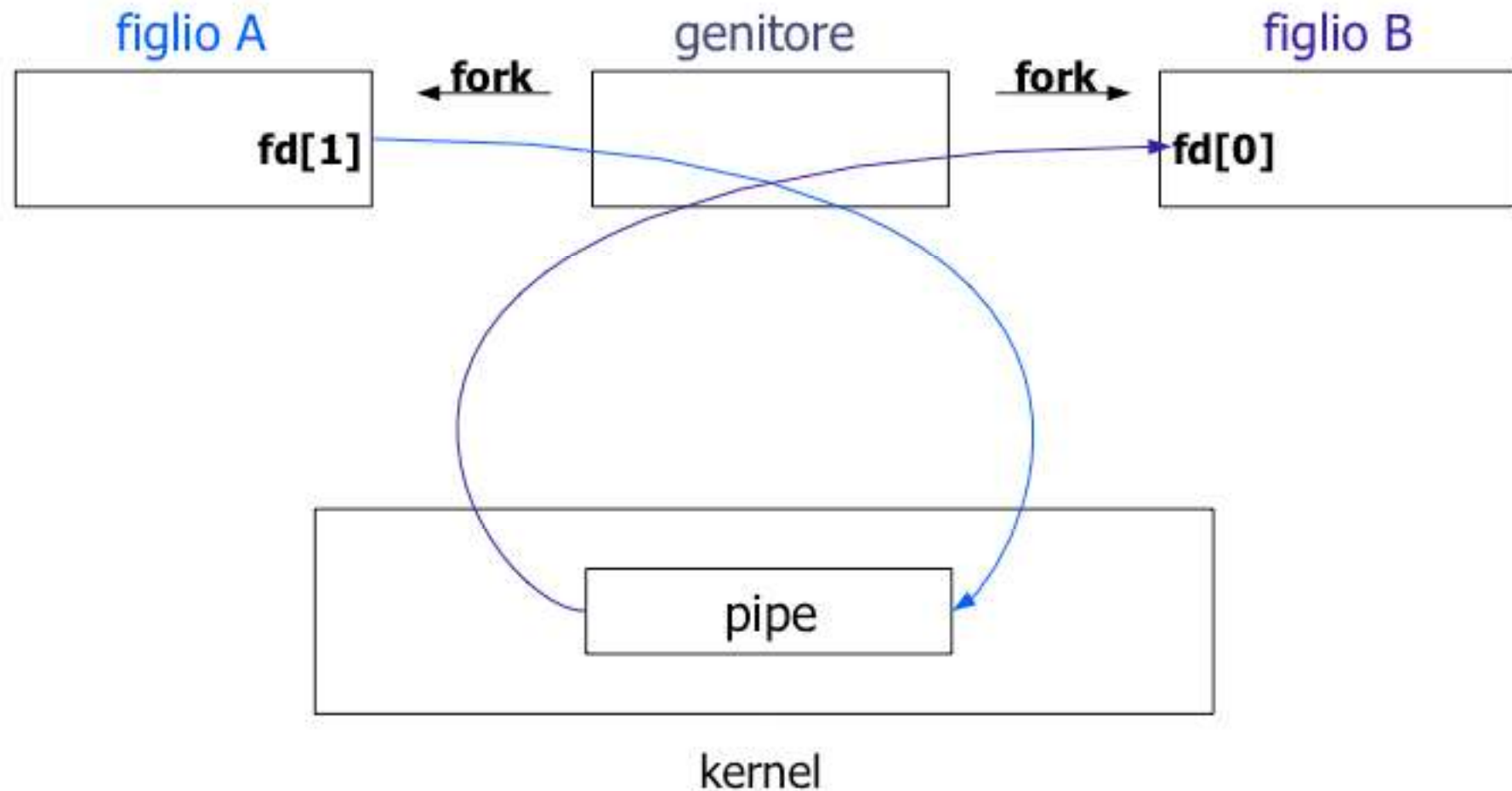
- 1 Il genitore crea una pipe usando la funzione `pipe`;
- 2 Il genitore crea due figli con la funzione `fork`, dopodichè chiude entrambi i lati della pipe;
- 3 il figlio **scrittore** chiude il lato **in lettura** della pipe ed associa il proprio **stdout** al lato **in scrittura** della pipe;
- 4 il figlio **lettore** chiude il lato **in scrittura** della pipe ed associa il proprio **stdin** al lato **in lettura** della pipe;
- 5 ciascuno dei figli carica con una `exec` il proprio programma;
- 6 Al termine della comunicazione, lo scrittore e il lettore **chiudono il lato** della pipe **di loro competenza**.

Pipe tra due programmi



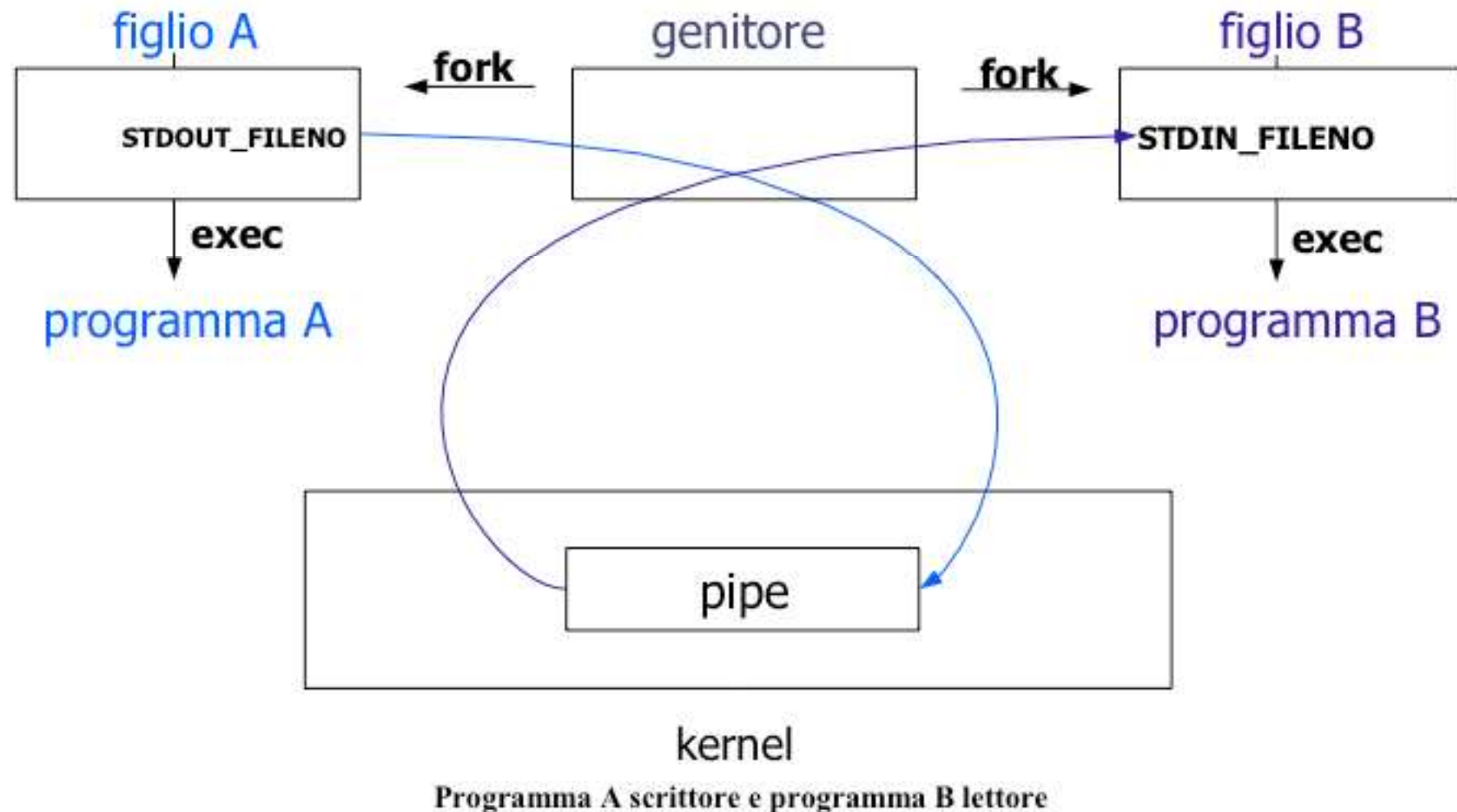
Pipe half-duplex dopo due chiamate a fork

Pipe tra due programmi



Figlio A scrittore e figlio B lettore

Pipe tra due programmi



```

/* implementa la pipe "who | wc" mediante la syscall pipe */

#include<unistd.h>
#include<stdio.h>
#include<sys/types.h>
#include<errno.h>

int main(void)
{
    int mypipe[2];
    pid_t pid1, pid2; /* 2 figli, 1 x ciascun programma */

    if (pipe(mypipe)<0) perror("pipe"), exit(1);

    if ((pid1=fork())<0) perror("fork"), exit(1);

    else if (pid1 == 0){ /* 1st child, reader */
        close(mypipe[1]);

        if(dup2(mypipe[0], STDIN_FILENO) != STDIN_FILENO)
            perror("dup2"), exit(1);

        close(mypipe[0]); /* opzionale */

        if (execl("/usr/bin/wc", "wc", NULL)<0)
            perror("execl"), exit(1);
    }
}

```

```

else{  /* pid1>0, parent */

    if ((pid2=fork())<0) perror("fork"), exit(2);

    else if (pid2 == 0){ /* 2nd child, writer */
        close(mypipe[0]); /* opzionale */

        if(dup2(mypipe[1], STDOUT_FILENO) != STDOUT_FILENO)
            perror("dup2"), exit(1);

        close(mypipe[1]); /* opzionale */

        if (execlp("who", "who", NULL)<0)
            perror("execlp"), exit(1);
    }
else{ /* pid1>0, pid2>0, parent */
    close(mypipe[0]); /* opzionale */
    close(mypipe[1]);

    /* attende per i 2 figli */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    exit(0);
}
}

```

popen e pclose

L'esecuzione di un comando da parte di un processo in modo che quest'ultimo possa riceverne l'output o inviargli l'input è una operazione molto comune, per la quale è dunque preferibile avere delle funzioni di più alto livello.

La libreria standard di IO fornisce invero le funzioni popen e pclose, che consentono al programmatore di evitare le azioni esplicite di creazione di una pipe, generazione di un figlio, chiusura del lato della pipe non utilizzato da parte dei processi scrittore e lettore, exec di una shell per l'esecuzione del comando e, infine, attesa della terminazione di quest'ultimo.

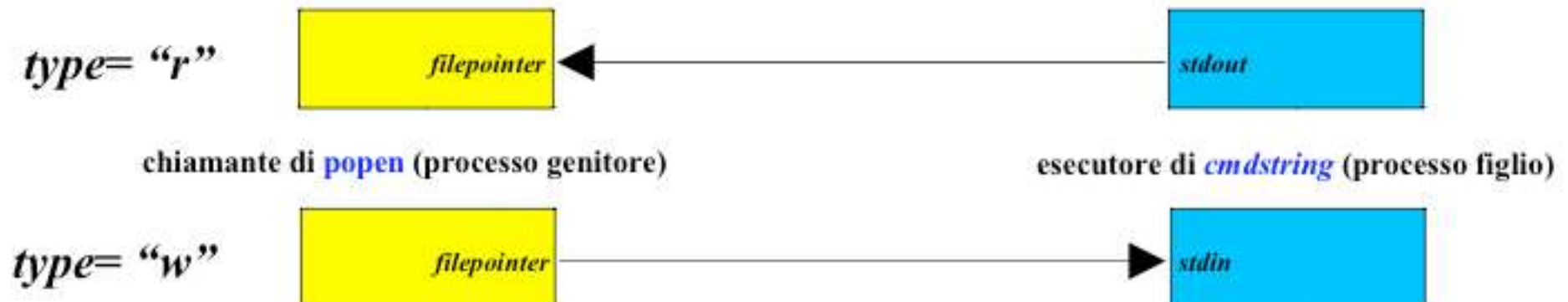
Più precisamente, popen crea una pipe, crea un figlio, chiude i lati non utilizzati della pipe ed esegue il comando; mentre pclose attende che l'esecuzione del comando sia terminata e chiude la pipe.

popen e pclose

```
#include <stdio.h>
```

```
FILE *popen (const char *cmdstring , const char *type );  
int pclose ( FILE *filepointer );
```

La funzione `popen` esegue il comando `cmdstring` e restituisce un puntatore a file per il processo chiamante. Se `type` è uguale a `"w"` il puntatore a file è connesso allo `stdin` del comando, se invece `type` è uguale a `"r"` tale puntatore è connesso allo `stdout`.



popen e pclose

Il comando *cmdstring* è eseguito come *sh -c cmdstring*, ossia la shell espande i caratteri speciali. Ad es., si può scrivere:

```
fp=popen("ls *.c", "r");           (eseguita da bourne shell)
```

La funzione *popen* restituisce un *puntatore a file* in caso di successo, il puntatore *NULL* in caso di errore.

La funzione *pclose* chiude lo stream di I/O standard e attende che il comando sia terminato, restituendo lo *stato di terminazione della shell*. Se la shell non può essere eseguita, lo stato di terminazione restituito da *pclose* equivale all'esecuzione da parte della shell di *exit(127)*.

Esempio: pager

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

#define MAXLINE 4096
#define PAGER "${PAGER:-more}" /* var. d'ambiente, o default */

int main(int argc, char *argv[ ])
{
    char line[MAXLINE];
    FILE *fpin *fpout;

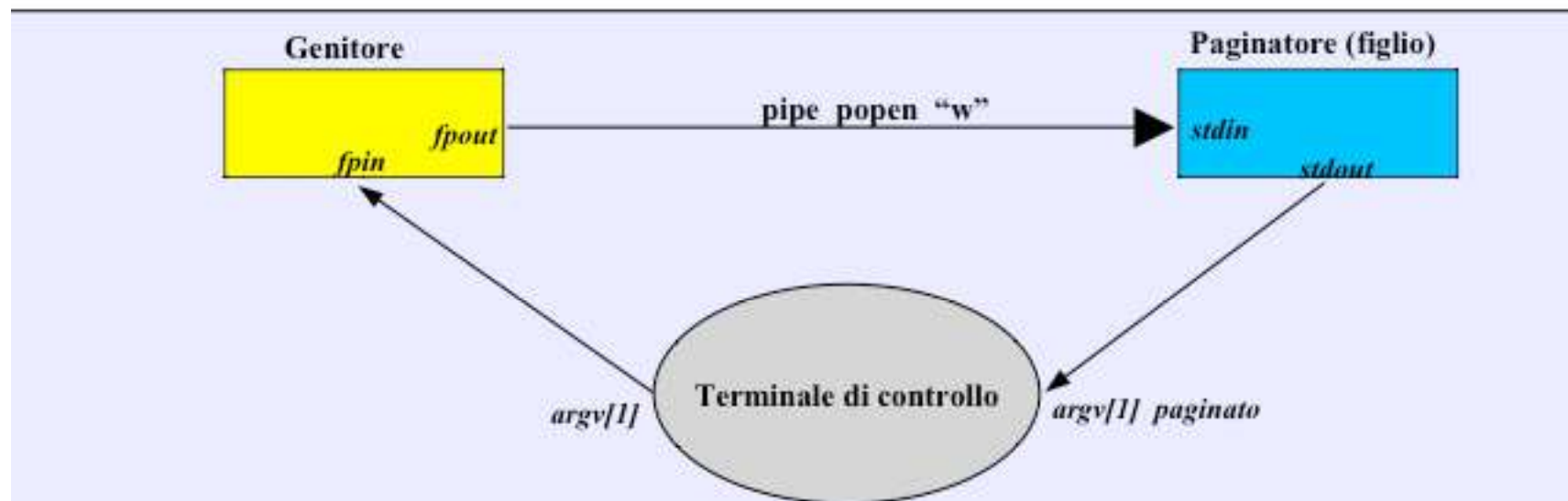
    if (argc != 2)
        printf("usage: %s <pathname>", argv[0]), exit(1);

    if ( (fpin = fopen(argv[1], "r")) == NULL)
        perror("fopen"), exit(1);

    /* crea un figlio "paginatore", collegandosi al suo stdin */
    if ( (fpout=popen(PAGER, "w") ) == NULL)
        perror("popen"), exit(1);
```

Esempio: pager

```
/* copia argv[1] al paginatore */  
while (fgets(line, MAXLINE, fpin) != NULL)  
    if (fputs(line, fpout) == EOF)  
        perror("fputs"), exit(1);  
  
if (ferror(fpin))  
    perror("fgets"), exit(1);  
  
if (pclose(fpout) == -1)  
    perror("pclose"), exit(1);  
  
exit(0);  
}
```



FIFO (o named pipe)

- Le pipe possono essere utilizzate solo se due processi hanno un “antenato” comune
 - Un processo crea la pipe e qualche discendente la usa.
- Le FIFO possono essere utilizzate per consentire la **comunicazione tra due processi arbitrari**.
 - Devono condividere solo il “nome” della FIFO

FIFO (named pipe)

I file speciali **FIFO** (**pipe con nome**) consentono di superare alcune delle limitazioni delle pipe. Essi infatti, rispetto a queste ultime, offrono i seguenti vantaggi:

- una volta creati, esistono nel file system fin tanto che non vengono esplicitamente cancellati;
- possono essere usati da processi che **non hanno un comune antenato**.

I file FIFO possono essere **creati** in due modi:

- attraverso la shell, con il **comando mkfifo**;
- all'interno di un programma, con la chiamata alla **funzione mkfifo**.

Una volta creato un file FIFO, su di esso si possono effettuare le **operazioni usuali di IO** su file (**open, read, write, close, ...**)

FIFO (o named pipe)

- Dopo la creazione della FIFO, puo' essere usata come “un file”
 - utilizzando open, read, write, close
 - NON la lseek
- E' possibile che piu' processi scrivano sulla stessa FIFO
 - Se il numero di byte scritti sulla FIFO e' inferiore a PIPE_BUF, le scritture sono “atomiche”
- L'utilizzo di O_NONBLOCK consente di non bloccare le operazioni di open/read/write.
 - Attenzione ad errori e SIGPIPE!

FIFO (named pipe)

Come per le pipe, se si esegue una `write` su di un file FIFO che nessun processo ha aperto `in lettura`, è generato il segnale `SIGPIPE`.

E' comune la situazione in cui più processi scrivono su di uno stesso file FIFO: affinché i dati non si mischino è necessario utilizzare `operazioni atomiche di scrittura`.

Come per le pipe, la costante `PIPE_BUF` stabilisce il massimo numero di byte che possono essere scritti in `maniera atomica` in un file FIFO.

FIFO

```
int mkfifo(char* pathname, mode_t mode);
```

- crea un FIFO dal **pathname** specificato
- la specifica dell'argomento **mode** è identica a quella di **open**, **creat** (mode codifica i permessi di accesso al file mediante un numero ottale, ad esempio 0644 = rw-r--r--)
- **Come funziona un FIFO?**
 - una volta creato un FIFO, le normali chiamate **open**, **read**, **write**, **close**, possono essere utilizzate per leggere il FIFO
 - il FIFO può essere rimosso utilizzando **unlink**
 - le regole per i diritti di accesso si applicano come se fosse un file normale

FIFO: open

- **Chiamata open**
 - File aperto senza flag **O_NONBLOCK**
 - Se il FIFO è aperto in sola lettura, la chiamata **si blocca** fino a quando un altro processo non apre il FIFO in scrittura
 - Se il FIFO è aperto in sola scrittura, la chiamata **si blocca** fino a quando un altro processo non apre il FIFO in lettura
 - File aperto con flag **O_NONBLOCK**
 - Se il FIFO è aperto in sola lettura, la chiamata **ritorna immediatamente**
 - Se il FIFO è aperto in sola scrittura, e nessun altro processo lo ha aperto in lettura, la chiamata **ritorna un messaggio di errore**

FIFO: write

- **Chiamata write**

- se nessun processo ha aperto il file in lettura viene generato un segnale **SIGPIPE**:
 - ignorato/catturato: write ritorna **-1** e **errno=EPIPE**
 - azione di default: terminazione

- **Atomicità**

- Quando si scrive su un pipe, la costante **PIPE_BUF** (in genere pari a 4096, vedi `/usr/include/linux/limits.h`) specifica la dimensione del buffer del pipe
- Chiamate **write** di dimensione inferiore a **PIPE_BUF** vengono eseguite in modo atomico
- Chiamate **write** di dimensione superiore a **PIPE_BUF** possono essere eseguite in modo non atomico
- La presenza di piu' scrittori può causare interleaving tra chiamate **write** distinte

Operazioni e Modalità

Operazione corrente	Status del descrittore complementare	Comportamento modalità bloccante	Comportamento modalità non bloccante
apertura FIFO in sola lettura	FIFO aperta in scrittura	ritorna con successo	ritorna con successo
	FIFO chiusa in scrittura	blocca finchè la FIFO è aperta in scrittura	ritorna con successo
apertura FIFO in sola scrittura	FIFO aperta in lettura	ritorna con successo	ritorna con successo
	FIFO chiusa in lettura	blocca finchè la FIFO è aperta in lettura	ritorna con l'errore ENXIO
lettura da pipe o FIFO vuote	pipe o FIFO aperta in scrittura	blocca finchè sono immessi dati o il lato in scrittura viene chiuso	ritorna con l'errore EAGAIN
	pipe o FIFO chiusa in scrittura	ritorna col valore 0 (fine del file)	ritorna col valore 0 (fine del file)
scrittura su pipe o FIFO	pipe o FIFO aperta in lettura	Se #byte<= PIPE_BUF, scrive in modo atomico, bloccando se non c'è spazio disponibile. Se #byte>PIPE_BUF scrive in modo non atomico.	Se #byte<=PIPE_BUF, scrive in modo atomico, ritornando con EAGAIN se non c'è spazio disponibile. Se #byte>PIPE_BUF e non c'è almeno 1 byte disponibile, ritorna con EAGAIN; altrimenti scrive solo ciò che può.
	pipe o FIFO chiusa in lettura	genera SIGPIPE	genera SIGPIPE

Esempio: comunicazione FIFO

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define MAX_BUF_SIZE 1000

int main(int argc, char *argv[]){

int fd, ret_val, count, numread;
char buf[MAX_BUF_SIZE];

/* Create the named - pipe */
ret_val = mkfifo("miafifo", 0666);
if ((ret_val == -1) && (errno != EEXIST)) {
    perror("Error creating the named pipe");
    exit (1); }

/* Open the pipe for reading */
fd = open("miafifo", O_RDONLY);

/* Read from the pipe */
numread = read(fd, buf, MAX_BUF_SIZE);
buf[numread] = '\0';
printf("Server : Read From the pipe : %s\n", buf);

/* Convert to the string to upper case */
count = 0;
while (count < numread) {
    buf[count] = toupper(buf[count]);
    count++;
}
printf("Server : Converted String : %s\n", buf);
}
```

Codice Server

Esempio: comunicazione FIFO

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

Codice Client

```
int main(int argc, char *argv[])
{
    int fd;

    /* Check if an argument was specified. */

    if (argc != 2) {
        printf("Usage : %s <string to be sent to the server>\n", argv[0]);
        exit (1);
    }

    /* Open the pipe for writing */
    fd = open("miafifo", O_WRONLY);

    /* Write to the pipe */
    write(fd, argv[1], strlen(argv[1]));
}
```

Esempio: comunicazione FIFO

Esempio esecuzione:

- `./servFifo &`
- `./clientFifo prova`
- Server : Read From the pipe : prova
- Server : Converted String : PROVA

Esercizio 1

- Scrivere un programma C che crea un figlio
 - il **padre** invia al figlio 10 numeri interi casuali al ritmo di uno al secondo, e poi termina
 - il **figlio** riceve i numeri dal padre e li stampa sul terminale
 - il **figlio** termina dopo aver ricevuto il decimo numero
- I due processi comunicano tramite una pipe
- Opzionale: modificare il figlio in modo che termini dopo aver ricevuto 5 interi

Esercizio 2

- Scrivere un programma C che crea un figlio
 - il **padre** entra in un ciclo in cui legge da terminale un numero intero x e manda al figlio il numero x^2
 - il **padre** esce dal ciclo e termina quando l'utente immette il numero 0
 - il **figlio** riceve i numeri dal padre e li stampa sul terminale
 - il **figlio** termina quando riceve 0 dal padre
- I due processi comunicano tramite una pipe